ADR-002: Architektúra systému Teamify

Oct 29, 2026 · @Ogi

Teamify implementujeme ako modulárny monolit nad jednou databázou PostgreSQL, s REST API a WebSocketmi na tom istom serveri.

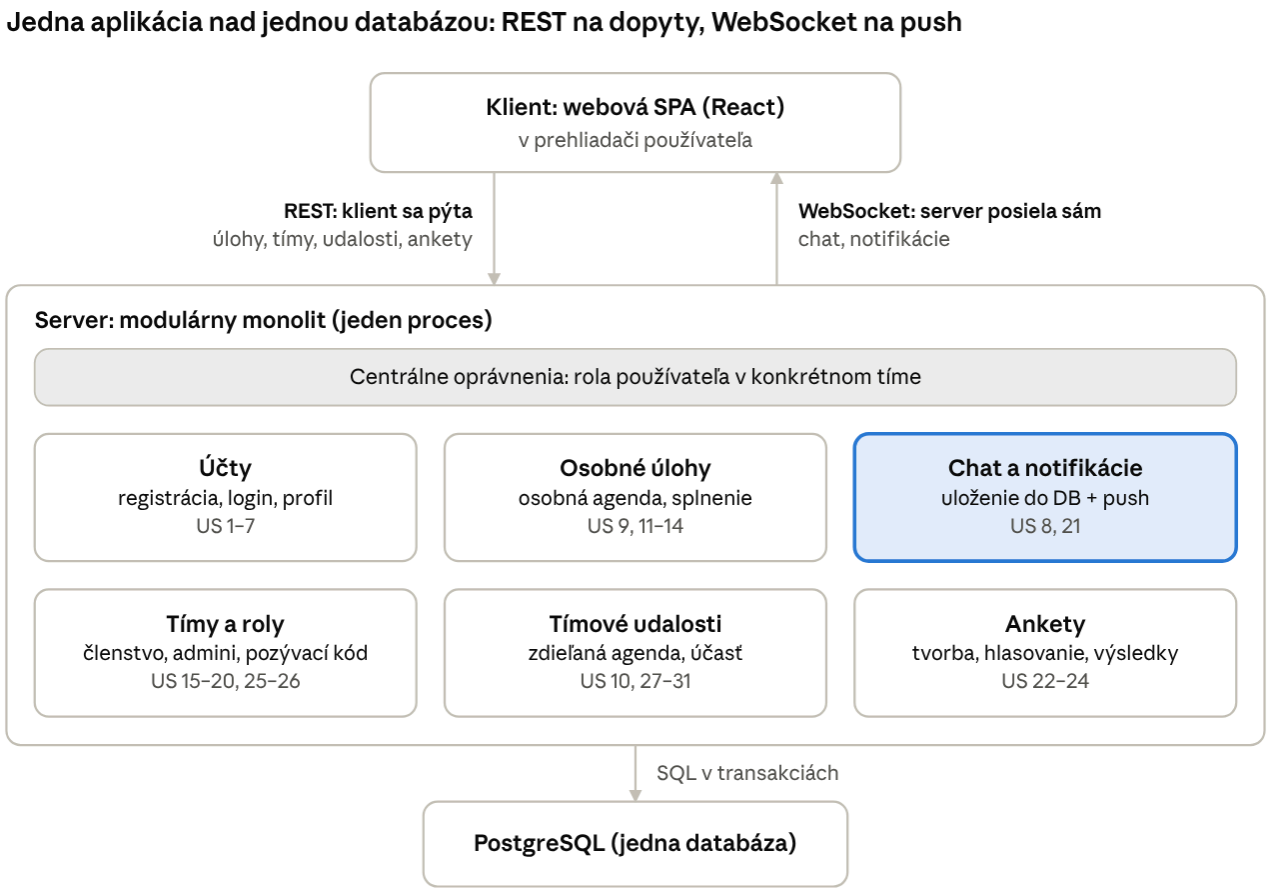


schéma systému Teamify · klient, server so 6 modulmi, databáza

Zvýraznený modul posiela správy v reálnom čase cez WebSocket; všetky ostatné požiadavky idú cez REST a pred každým modulom prejdú kontrolou roly v tíme.

Metadáta

Pole	Hodnota
Stav	Navrhnuté (po schválení tímom zmeniť na Prijaté)
Dátum	29. 10. 2026
Rozhodovatelia	Lukáš Ogurčák a projektový tím

Pole	Hodnota
Rozsah	celková architektúra Teamify: kde beží logika, ako sú uložené dáta, ako funguje real-time
Podklady	Teamify.txt (zadanie, 31 user stories), diskusia o architektúrach A / B / C

Kontext

Teamify spája osobné úlohy s tímami, ktoré majú zdieľanú agendu udalostí, chat, ankety a notifikácie (31 user stories, roly člen a admin). Architektúru najviac ovplyvňujú tri veci:

- **Real-time:** správy v chate (US 21) a notifikácie (US 8, 16, 18) musí server poslať sám, bez toho, aby sa klient pýtal.
- **Roly v tíme sa menia za behu** (US 19, 20): kto je admin, sa mení počas používania a tím nesmie ostať bez admina.
- **Kaskády pri mazaní:** zmazanie tímu (US 26) alebo účtu (US 7) sa dotkne udalostí, ankiet, hlasov a správ.

Obmedzenia: malý študentský tím, čas jedného semestra, offline režim zadanie nevyžaduje.

Rozhodnutie

Volíme variant A, modulárny monolit: jedna aplikácia rozdelená na moduly nad jednou databázou PostgreSQL.

- **Klient:** webová SPA v Reacte; REST na čítanie a zmeny dát, WebSocket na push chatu a notifikácií.
- **Moduly:** Účty, Osobné úlohy, Tímy a roly, Tímové udalosti, Chat a notifikácie, Ankety.
- **Oprávnenia:** jedna centrálna kontrola roly používateľa v konkrétnom tíme pred každým modulom.
- **Stack:** backend v tom, čo tím už ovláda (Spring Boot, NestJS alebo Django) + PostgreSQL + React.
- **Zjednodušenia:** jedna inštancia servera bez Redisu; notifikácia sa vždy uloží do DB a online používateľovi ide navyše cez WebSocket; do tímu sa vstupuje pozývacím kódom.

Zvažované možnosti

A vyhral, lebo je najjednoduchší variant, ktorý pokryje všetkých 31 user stories: väčšina bolestí B v ňom nevznikne a bolesti C rieši jedna transakcia a obmedzenia v databáze.

	A: Modulárny monolit	B: Mikroslužby s event busom	C: Serverless / BaaS
Popis	jedna aplikácia s modulmi, jedna DB, REST + WebSocket	služby s vlastnými DB, API gateway, broker	Supabase / Firebase, cloudové funkcie, pravidlá v DB
Rýchlosť vývoja	vysoká	nízka	najvyššia
Škálovanie	jedna inštancia; viac až s Redis pub/sub	každá služba zvlášť	rieši poskytovateľ
Konzistencia dát	silná, transakcie	eventuálna, sagy	závisí od pravidiel v DB
Real-time	WebSocket na vlastnom serveri	notifikačná služba počúva broker	hotové odbery zmien v DB
Prevádzková náročnosť	nízka	vysoká	najnižšia
Najväčší pain	správa WebSocket spojení	dual write → outbox pattern	posledný admin a kaskády v security rules
Výsledok	**zvolené**	zamietnuté	záložná voľba

B zamietnuté: infraštruktúra (gateway, broker, viac databáz) a distribuovaná konzistencia (outbox, sagy, zastaraná rola v JWT) by zabrali viac času než samotné user stories. Nezávislé škálovanie služieb Teamify nepotrebuje.

C záložná voľba: najrýchlejšie k funkčnej aplikácii, ale pravidlo posledného admina a kaskádové mazanie sa v databázových pravidlách ťažko píšu a testujú a viažeme sa na poskytovateľa (vendor lock-in). Ak by sme ho volili, tak Supabase, nie Firestore.

Dôsledky

Získame	Platíme
invarianty a kaskády v jednej transakcii	celá aplikácia sa nasadzuje naraz

Získame	Platíme
rýchly vývoj, jeden repozitár	disciplínu pri hraniciach modulov
jeden server a jedna DB na prevádzku	viac inštancií až s Redis pub/sub
plnú kontrolu nad oprávneniami a dátami	WebSocket spojenia spravujeme sami

Riziká a mitigácie

Najťažšie nebudú obrazovky ani CRUD, ale real-time doručovanie, súbežnosť a invarianty.

Riziko	Prejav	Mitigácia
správa WebSocket spojení	viac zariadení, výpadok siete, zmeškané správy	notifikácie vždy v DB; po reconnecte klient dotiahne zmeškané cez REST
race condition pri adminoch	dvaja admini si v tej istej sekunde odoberú práva → tím bez admina	transakcia so `SELECT … FOR UPDATE` a kontrola počtu adminov
zabudnutá kontrola oprávnení	nový endpoint bez kontroly roly	centrálna kontrola pred modulmi, testy pre každú rolu
dvojité hlasovanie	dvojklik alebo dve karty prehliadača	unikátne obmedzenie (anketa, používateľ) v DB
zmazanie účtu	správy a hlasy odkazujú na neexistujúceho používateľa	„Zmazaný používateľ“ namiesto mazania správ
časové pásma	udalosť sa ukáže v zlom čase	časy v UTC, prevod až v klientovi
rozmazané hranice modulov	modul číta cudzie tabuľky	moduly sa volajú cez rozhranie, nie cez cudzie tabuľky

Otvorené otázky a prehodnotenie

Zadanie necháva tri veci otvorené; treba ich potvrdiť s vyučujúcim pred návrhom DB schémy.

- [ ] Vstup do tímu (US 16): pozývací kód (náš predpoklad), pozvánka, alebo žiadosť o vstup?
- [ ] Posledný admin opúšťa tím (US 18): musí vymenovať nástupcu, alebo sa tím zmaže?
- [ ] Zmazanie účtu (US 7): čo s jeho udalosťami, hlasmi a tímami, kde je jediným adminom?

- [ ] Schválenie ADR tímom a zmena stavu na Prijaté.

Rozhodnutie prehodnotíme, ak:

- predmet bude výslovne vyžadovať distribuovaný systém → variant B;
- predmet nebude hodnotiť návrh backendu a prioritou bude rýchlo funkčná aplikácia → variant C cez Supabase;
- bude treba viac inštancií servera → doplniť Redis pub/sub, architektúra ostáva.

Ďalšie kroky: diagram modulov a ich prepojení, návrh databázovej schémy.